

Iiwa Became an MLB Legend: Model Predictive Control for Reactive Batting in Simulation

1st Kota Uchida

Massachusetts Institute of Technology
Cambridge, USA
kotau@mit.edu

2nd Hayato Iwami

Massachusetts Institute of Technology
Cambridge, USA
hayatoi@mit.edu

Abstract—Striking a fast ball with a bat requires predicting its motion and generating a high-speed swing that respects joint and torque limits. Recent work has shown impressive table tennis on quadrupeds using whole-body model predictive control (MPC), but these methods are complex and tailored to floating-base robots. We instead study an arm-only setting: a simulated fixed-base KUKA IIWA arm holding a bat. Assuming the ball trajectory is known from the simulator, we adapt the Bézier-based kinematic MPC from prior work to plan joint trajectories that bring the bat to a desired pose, velocity, and normal at impact while staying near a rest posture and within joint limits. The resulting desired joint states are tracked with an inverse-dynamics controller in Drake. We evaluate batting performance across ball speeds and target locations, and report hit rate, landing error, and solve time.

I. INTRODUCTION

Hitting a moving ball with a racket or bat is a prototypical dynamic manipulation task. A controller must predict the ball trajectory, choose a contact point and timing, and execute a fast swing that achieves the desired contact state while respecting joint and torque limits. Robotic systems have achieved impressive results in table tennis and related sports, including recent whole-body MPC approaches on quadrupeds that jointly optimize base motion, foot contacts, and racket motion. However, such formulations are high dimensional, rely on detailed contact models for the legs, and are not directly suited to common fixed-base manipulators.

In this project we ask how much of the benefit of whole-body MPC can be retained in an arm-only setting. We consider a simulated KUKA IIWA arm rigidly mounted to the ground and equipped with a bat. A ball is launched toward the robot with known trajectory from the simulator, and the goal is to strike it so that the ball lands in a desired target region. We adapt the Bézier-based kinematic MPC module from prior table tennis work to plan short-horizon joint trajectories that meet impact constraints on the bat’s pose, translational velocity, and surface normal while keeping the arm near a nominal rest posture and within joint limits. The planned joint positions, velocities, and accelerations are tracked using an inverse-dynamics controller in Drake, and we evaluate

the resulting batting performance and computation time in simulation.

II. RELATED WORK

A. Robotic Table Tennis and Batting

Robotic racket sports have been studied extensively, with most systems using fixed-base arms or gantry robots to play table tennis. Early work focused on arm-only systems that track the ball and move the paddle to a predicted contact point using inverse kinematics and heuristic swing profiles, often with limited consideration of joint limits or contact dynamics. More recent systems employ high-speed perception and learning-based control to achieve rallies with human players and to handle a range of spins and return targets. Nguyen et al. propose a whole-body MPC framework for a quadruped with a racket, jointly optimizing base motion, foot contacts, and paddle motion for spin-aware table tennis. Our work is closest in spirit to this approach, but we restrict attention to a fixed-base KUKA IIWA with a bat and study how a simplified, arm-only version of their kinematic MPC performs in a batting task.

B. MPC and Trajectory Optimization for Dynamic Manipulation

Trajectory optimization and MPC have become standard tools for dynamic manipulation tasks such as catching, throwing, and hitting. Prior course projects and research systems have used optimization-based planners to move a manipulator to an intercept pose for catching or to plan release motions for throwing, often assuming privileged information about the object trajectory and focusing on kinematic feasibility. In dexterous batting and racket sports, additional work has modeled ball–racket or ball–bat impacts to predict post-contact trajectories and to design suitable contact velocities. Our method follows this line of optimization-based control: we adopt the Bézier-parameterized kinematic MPC from prior table tennis work as the core trajectory generator, but apply it to a simplified arm-only setting and couple it with an inverse-dynamics controller in Drake for execution.

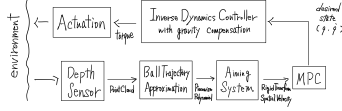


Fig. 1. Schematic of our system

III. APPROACH

Our approach follows the structure suggested by prior table-tennis work: an *aiming* module that chooses a desired paddle state at impact, a *kinematic MPC* that generates a short-horizon joint trajectory realizing this state, and a *low-level controller* that tracks the planned trajectory on the KUKA IIWA model in Drake. Figure 1 summarizes the overall architecture.

A. Task and System Overview

We consider a simulated KUKA IIWA arm rigidly mounted to the ground and holding a long bat. A ball is launched toward the robot with known initial position and velocity; Drake’s physics engine then provides the ground-truth ball trajectory. Our controller receives this predicted trajectory and must generate a swing that strikes the ball once and sends it toward a user-specified target region on the far side of the workspace. We deliberately assume perfect knowledge of the ball trajectory and ignore perception noise and model error in order to focus on trajectory generation and control.

The robot configuration is described by a joint vector $q \in \mathbb{R}^{n_q}$ and joint velocity $\dot{q}$. The end-effector (bat) pose in the world frame is given by a homogeneous transform $X_{WB}(q)$, and we denote the bat’s translational velocity and angular velocity by $v_B(q, \dot{q}) \in \mathbb{R}^3$ and $\omega_B(q, \dot{q}) \in \mathbb{R}^3$, respectively. We also define a unit normal vector $n_B(q)$ pointing roughly along the striking surface of the bat.

B. Aiming and Desired Strike State

Given the predicted ball trajectory $p_b(t)$ from the simulator and a desired landing region on the ground, the aiming module constructs a *desired strike state* for the bat. Following the spirit of Nguyen *et al.*, we specify this state by

$$p_{\text{des}}, \quad v_{\text{des}}, \quad n_{\text{des}}, \quad t_{\text{hit}},$$

where $p_{\text{des}} \in \mathbb{R}^3$ is the desired bat position at impact, $v_{\text{des}} \in \mathbb{R}^3$ is the desired translational velocity of the bat’s contact point, n_{des} is the desired surface normal, and t_{hit} is the chosen impact time.

In our implementation, p_{des} is selected as a point on the ball trajectory inside a predefined “hitting volume” in front of the robot. The time t_{hit} is the corresponding time along $p_b(t)$. We then choose n_{des} to point roughly from the ball toward the target region, and set v_{des} to be a fixed magnitude in that direction. This simple heuristic ignores spin and detailed impact modeling, but provides a consistent way to generate strike states for different ball trajectories and targets.

The aiming module is implemented as a Drake LeafSystem. It takes the predicted ball trajectory and the current IIWA state

as inputs and outputs (i) the desired end-effector transform X_{des} corresponding to p_{des} and n_{des} , and (ii) a desired spatial velocity twist for the bat that encodes v_{des} .

C. Arm Kinematic MPC with Bézier Trajectories

The core of our controller is a kinematic model predictive controller that plans a joint-space trajectory over a short time horizon $[0, T]$. Following Nguyen *et al.*, we parameterize this trajectory using Bézier curves. Let $\tau = t/T \in [0, 1]$ be the normalized time, and let $q_c[0], \dots, q_c[N_c] \in \mathbb{R}^{n_q}$ be the control points. The planned joint trajectory is

$$q(\tau) = B(q_c, \tau) = \sum_{k=0}^{N_c} \beta_k(\tau) q_c[k],$$

where $\beta_k(\tau)$ are the Bernstein basis polynomials. We obtain $\dot{q}(\tau)$ and $\ddot{q}(\tau)$ by differentiating the Bézier curve.

The decision variables of the optimization are the control points q_c , together with auxiliary variables q_s and $\dot{q}_s$ representing the joint position and velocity at impact time $\tau_s = t_{\text{hit}}/T$. The optimization problem is

$$\min_{q_c, \dot{q}_c, q_s, \dot{q}_s} W_a f_a(\ddot{q}_c) + W_r f_r(q_{\text{rest}}, q_c) \quad (1)$$

subject to

$$B(q_c, 0) = q_0, \quad (2)$$

$$B'(q_c, 0) = \dot{q}_0, \quad (3)$$

$$q_{\min} \leq B(q_c, \tau_i) \leq q_{\max} \quad \forall \tau_i \in \mathcal{T}, \quad (4)$$

$$B(q_c, \tau_s) = q_s, \quad (5)$$

$$B'(q_c, \tau_s) = \dot{q}_s, \quad (6)$$

$$K_p(q_s) = p_{\text{des}}, \quad (7)$$

$$J_p(q_s) \dot{q}_s = v_{\text{des}}, \quad (8)$$

$$\|K_n(q_s) - K_p(q_s) - n_{\text{des}}\|_2^2 \leq \varepsilon_n, \quad (9)$$

where $q_0, \dot{q}_0$ are the current joint state, $K_p(\cdot)$ maps joint positions to the bat contact point position, $J_p(\cdot)$ is the corresponding Jacobian, and $K_n(\cdot)$ returns a point along the bat normal. The set $\mathcal{T}$ contains a finite set of sample times along the horizon at which we enforce joint limits. In contrast to the original whole-body formulation, we do not impose any foot-position constraints because the base of the IIWA is fixed.

The cost terms f_a and f_r encourage smooth swings and keep the arm near a comfortable rest posture. Following Nguyen *et al.*, we define

$$f_r(q_{\text{rest}}, q_c) = \sum_{k=0}^{N_c} (q_c[k] - q_{\text{rest}})^\top W_j (q_c[k] - q_{\text{rest}}), \quad (10)$$

$$f_a(\ddot{q}_c) = \sum_{k=0}^{N_c-2} \ddot{q}_c[k]^\top W_j \ddot{q}_c[k], \quad (11)$$

where $\ddot{q}_c[k] = q_c[k+2] - 2q_c[k+1] + q_c[k]$ is a second-difference proxy for joint acceleration and W_j is a joint-space weighting matrix. Introducing $\ddot{q}_c$ as an auxiliary

variable allows us to keep the cost quadratic in the decision variables.

We solve this quadratic program using Drake’s mathematical programming interface and a sparse QP solver (OSQP) at each control cycle. Only the first portion of the planned trajectory is actually executed; at the next time step, we replan with updated initial conditions, forming a receding-horizon MPC scheme.

D. Low-Level Tracking in Drake

The kinematic MPC produces time-parameterized desired joint states $(q_{\text{des}}(t), \dot{q}_{\text{des}}(t), \ddot{q}_{\text{des}}(t))$ over the horizon. To execute this trajectory on the simulated robot, we use an inverse-dynamics-based joint controller provided by Drake. At each control update, the controller receives the estimated current joint state $(q, \dot{q})$ and the desired state at the current time, and computes joint torques τ that approximately satisfy

$$M(q)\ddot{q}_{\text{des}} + C(q, \dot{q}) + g(q) = \tau + J^T(q)f_{\text{contact}},$$

with additional PD feedback terms to reject tracking errors. Here M is the mass matrix, C collects Coriolis and damping effects, and g is gravity.

This separation between a purely kinematic MPC and a standard inverse-dynamics controller keeps the optimization problem relatively small while still producing physically plausible swings in simulation. It also mirrors a common structure in industrial robot control, making our design easier to transfer to other fixed-base manipulators in future work.

IV. RESULTS

In this section we evaluate our arm-only MPC controller in the simulated batting task. We first describe the metrics and experimental setup, and then report quantitative results across different ball speeds and target locations.

A. Metrics

We use the following metrics:

- **Hit rate:** the fraction of trials in which the bat makes at least one contact with the ball before it passes the robot.
- **Landing error:** the Euclidean distance between the ball’s landing point on the ground and the center of the desired target region. We also report the fraction of trials that land inside a radius- R neighborhood of the target.
- **Flight distance:** the horizontal distance from the robot (or home plate) to the ball’s landing point, as a simple proxy for “power” or home-run potential.
- **Exit velocity:** the norm of the ball’s translational velocity immediately after the first bat-ball contact.
- **Strike state error:** the position, velocity, and normal errors between the desired and actual bat state at impact.
- **MPC solve time:** the average and maximum time required to solve the kinematic MPC at each control cycle.

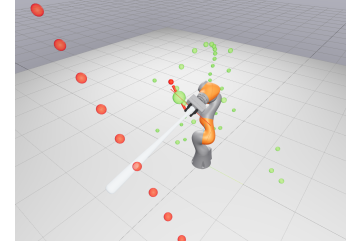


Fig. 2. MPC trajectory

B. Experimental Setup

We evaluate the controller across a range of incoming ball speeds and several different target locations. For each condition we run a fixed number of trials with randomized initial ball trajectories inside a prescribed range. The controller replans at 50 Hz with a prediction horizon of $T = 2.0$ s and uses N Bézier control points. We log all ball and robot states at each simulation step and compute the metrics defined above.

C. Quantitative Results

In order to qualitatively evaluate the performance, we made multiple trials to calculate the statistics.

About hit rate, 3 out of 10 times succeed when starting from random different initial configuration of iiwa. It was difficult to hit the ball when the trajectory of the end-effector has to go up and down, probably because this motion causes the inertia to the body, which might exceed the upper bound of the torque.

The angular error of the ball after strike was recorded for 4 trials. From this experiment, the standard deviation was 10.4 deg. Note that this excludes the cases where the hit failed.

The running time of the MPC was recorded over 10 trials, including both successful cases and failure cases. The hitting pose was calculated in 3.30sec, and the MPC cycle in 2.65 sec per timestep. This implies that our implementation is not real-time.

V. DISCUSSION

The results highlight both the strengths and limitations of the proposed arm-only MPC approach.

A. Effectiveness of the Bézier MPC

We find that the Bézier-based kinematic MPC can generate smooth, natural swing motions that approximately realize the desired bat pose and velocity at impact.

In particular, the controller tends to exploit the redundancy of the arm to keep joints near the rest posture while aligning the bat with the target direction. Figure 2 shows the point of the trajectory calculated with MPC.

B. Failure Modes

The main failure modes we observe are:

- Late or early swings when the ball speed becomes too high, leading to misses or glancing contacts.
- Joint-limit-induced awkward postures for extreme target locations, which reduce exit velocity and accuracy.

- Occasional MPC timeouts or degraded solutions when the optimization problem becomes ill-conditioned.

C. Limitations and Future Work

Our study intentionally ignores perception noise, model uncertainty, and spin, and is restricted to a fixed-base arm in simulation. Future work includes coupling the controller with a real perception pipeline, incorporating a more accurate impact model and spin-aware aiming, and extending the formulation to real hardware. We also plan to explore learned warm-start policies or reduced-order models to further improve real-time performance at higher ball speeds.

REFERENCES

REFERENCES

- [1] D. Nguyen, Z. Zaidi, K. Karol, J. Hodgins, and Z. Xie, “Whole Body Model Predictive Control for Spin-Aware Quadrupedal Table Tennis,” *arXiv preprint arXiv:2510.08754*, 2025.
- [2] D. Nguyen, K. D. Cancio, and S. Kim, “High Speed Robotic Table Tennis Swinging Using Lightweight Hardware with Model Predictive Control,” *arXiv preprint arXiv:2505.01617*, 2025.
- [3] P. Tobuschat, H. Ma, D. Büchler, and M. Muehlebach, “Data-Efficient Online Learning of Ball Placement in Robot Table Tennis,” *arXiv preprint arXiv:2308.14562*, 2023.
- [4] R. R. Burridge, A. A. Rizzi, and D. E. Koditschek, “Sequential Composition of Dynamically Dexterous Robot Behaviors,” *The International Journal of Robotics Research*, vol. 18, no. 6, pp. 534–555, 1999.
- [5] B. Cieslar, M. Zeng, and R. Tedrake, “CatchingBot: A Robotic System for Catching Objects in Flight,” MIT 6.4210 Final Project Report, 2023.
- [6] J. Zhang and D. Klahn, “Ping Pong Bot 2023,” MIT 6.4210 Final Project, 2022.
- [7] D. Yang and T. Wang, “Throwing in Drake,” MIT 6.4210 Final Project, 2020.